

Questo documento descrive la consegna 2 (finale) del progetto software individuale che costituisce una delle prove d'esame dell'insegnamento "Programmazione di sistemi embedded" (9 CFU, codice IN01122661) edizione 2025-26.

La consegna deve essere completata

**entro domenica 31 maggio 2026**

ri-caricando in Moodle il link al repository Git già utilizzato per la consegna 1. È necessario che il repository contenga la storia dei commit sia per la consegna 1 che per la consegna 2, preferibilmente in un unico branch; la consegna 2 è stata pensata in modo che sia possibile modificare ed evolvere il codice della consegna 1. Per il contenuto del repository valgono le indicazioni già date per la consegna 1 (presenza e contenuto del file README, assenza dei file generati da Android Studio durante il processo di build): si rimanda al relativo documento per maggiori dettagli.

### **Divieti**

Ciascuno studente deve completare individualmente il progetto software. Tra studenti non è consentito collaborare nella realizzazione, anche se sono consentiti degli scambi di idee.

È vietato l'uso di strumenti di IA generativa per creare documentazione (commenti al codice, tabelle, ecc.) e porzioni di codice sorgente superiori alle 10 righe. È vietato anche utilizzare, senza significative modifiche, porzioni di codice sorgente superiori alle 10 righe scritte da altri: questo divieto include il codice che si trova online. È permesso l'uso di strumenti di IA generativa per creare elementi grafici (immagini, icone, ecc.). È permesso l'uso di strumenti di IA generativa per avere feedback sul proprio codice (segnalazione di errori, suggerimenti per soluzioni alternative, ecc.) ma solo dopo averlo scritto, ed è vietato far generare all'IA codice per rispondere ai feedback (correzione degli errori, implementazione dei suggerimenti, ecc.).

È vietato utilizzare librerie che non fanno parte dei [framework di Android](#) e di [Jetpack](#) a meno che non si sia prima ottenuto il consenso scritto del docente.

### **Ore necessarie per il progetto software individuale**

"Programmazione di sistemi embedded" è un insegnamento da 9 CFU, quindi ciascuna studentessa e studente che voglia superare l'esame deve investire, nominalmente, 225 ore di lavoro (25 ore per CFU). L'impegno stimato per superare l'esame realizzando il progetto software individuale si ripartisce come segue.

| <b>Attività</b>                                                                                       | <b>Ore</b> |
|-------------------------------------------------------------------------------------------------------|------------|
| Partecipazione alle lezioni frontali                                                                  | 72         |
| Lavoro individuale di studio del materiale necessario per il progetto software e per la prova scritta | 80         |
| Lavoro individuale per il progetto software, consegna 1 (intermedia)                                  | 30         |
| Lavoro individuale per il progetto software, consegna 2 (finale)                                      | 43         |
| <b>TOTALE</b>                                                                                         | <b>225</b> |

## Descrizione della consegna

Si chiede di realizzare una app che costituisce un prototipo funzionante del gioco “[Simon](#)” (variante).

L'interfaccia utente deve **funzionare correttamente sia** quando il dispositivo Android si trova **in modalità portrait che** quando si trova **in modalità landscape**.

L'interfaccia utente deve **supportare almeno due lingue: italiano e inglese**.

L'interfaccia utente deve avere **tre schermate**, come descritto qui sotto in dettaglio.

### Lista delle Partite

Questa schermata, che è **la prima ad essere mostrata all'avvio dell'app**, è un'evoluzione della “Schermata 2” nella consegna precedente. La schermata mostra una lista dinamica dei dati sulle partite concluse dall'installazione dell'applicazione. L'ordine in cui sono visualizzate le partite può essere qualsiasi. Ciascun elemento della lista contiene i seguenti due elementi.

- Sulla sinistra, la **lunghezza massima di una sequenza riprodotta correttamente dal giocatore**.
- Sulla destra, la **sequenza di rettangoli completa in cui si è verificato il primo errore**. Dal **punto in cui si è verificato l'errore** (compreso) fino alla fine, **la sequenza deve essere visualizzata in un colore diverso**. Se la sequenza è troppo lunga per essere visualizzata interamente ne **viene mostrato solo l'inizio**, come nella consegna precedente.

Quando si clicca un elemento della lista, viene visualizzata la partita completa nella schermata **Dettaglio Partita**.

La Lista delle Partite contiene anche un **pulsante che porta alla Schermata di Gioco**; tale pulsante può essere di tipo convenzionale o un **floating action button**.

### Dettaglio Partita

È una schermata molto semplice, che **visualizza la partita con lo stesso aspetto della Lista delle Partite ma con più spazio a disposizione**. Da questa schermata **si esce con il tasto “Back” di sistema** (fisico, virtuale o *touch gesture*).

### Schermata di Gioco

Questa schermata è un'evoluzione della “Schermata 1” nella consegna precedente e contiene i seguenti elementi.

- a) **Matrice 3 righe per 2 colonne di rettangoli colorati**. Valgono le stesse specifiche della consegna precedente.
- b) **Area di testo multiriga non editabile** che visualizza la sequenza di rettangoli premuti fino a quel momento. Valgono le stesse specifiche di formato della consegna precedente.
- c) **Area pulsanti “Avvia partita”, “Pausa” e “Fine partita”**. La disposizione dei pulsanti all'interno dell'area può essere qualsiasi.

Disposizione degli **elementi della schermata in modalità portrait e landscape**: valgono le stesse specifiche della consegna precedente.

**Il pulsante “Avvia partita” è attivo solo quando si entra nella schermata**. Quando il pulsante viene premuto, **esso viene disattivato** e inizia, appunto, la partita, che procede a turni e con le **modalità del gioco Simon fisico**: prima **il computer propone una sequenza** di colori, poi il **giocatore deve replicarla** correttamente premendo i corrispondenti rettangoli nell'ordine giusto; se il giocatore riesce nell'intento, **il computer propone una nuova sequenza più lunga**

di 1, uguale alla precedente tranne nell'ultimo colore, che è nuovo. Si parte da una sequenza di lunghezza 1 e i colori vengono generati in maniera casuale. Quando il giocatore preme un rettangolo sbagliato, viene mostrata una segnalazione di errore (a scelta), la partita termina ma si rimane nella Schermata di Gioco in attesa che il giocatore prema il tasto "Back" di sistema. Durante una proposta da parte del computer l'area di testo multiriga rimane vuota. Quando è il turno del giocatore, l'area di testo visualizza la sequenza di rettangoli premuti. Sia durante la proposta da parte del computer che durante le interazioni del giocatore viene fornito un duplice feedback: visivo sul rettangolo del colore attivo (a scelta: può cambiare l'intensità del colore, può cambiare la forma, o altro) e uditivo (viene riprodotto un tono, diverso per ogni rettangolo).

Il pulsante "Pausa" è attivo solo mentre è in corso una proposta da parte del computer. Quando il pulsante viene premuto, esso si trasforma in un pulsante "Riprendi" e la riproduzione della proposta da parte del computer si ferma fino a nuova pressione.

Il pulsante "Fine partita" è attivo solo durante una partita, cioè dalla pressione di "Avvia partita" fino a quando il giocatore non sbaglia. Quando il pulsante viene premuto, si torna subito alla Lista delle Partite. Se al momento della pressione è in corso la presentazione della prima sequenza, cioè quella di lunghezza 1, l'app si comporta come se la partita non fosse mai iniziata e alla lista non viene aggiunto alcun elemento. In tutti gli altri casi, l'app si comporta come se il giocatore avesse commesso un errore nel punto della sequenza in cui ci si trovava (primo rettangolo qualora il giocatore non ne avesse ancora premuto alcuno) e alla lista viene aggiunta una partita in questo senso.

Quando viene premuto il tasto "Back" di sistema e la partita è terminata, si torna alla Lista delle Partite aggiungendo alla lista un nuovo elemento: la partita appena conclusa. Se la partita non è terminata, l'app si comporta come se fosse stato premuto il pulsante "Fine partita".

L'app deve gestire sia lo stato dell'istanza che lo stato persistente. In particolare, durante un cambio di configurazione (esempio: commutazione portrait/landscape) nella Schermata di Gioco deve

- continuare la riproduzione della sequenza da parte del computer, se in corso;
- essere preservata la sequenza di rettangoli premuti fino a quel momento.

Durante un cambio di configurazione è accettabile che un tono in corso di riproduzione venga troncato, purché la sequenza riprenda regolarmente dal tono successivo.

Per la Lista delle Partite deve essere preservato il contenuto della lista anche nel caso in cui l'app venga chiusa o il dispositivo riavviato. Per la Schermata di Gioco non è necessario, nelle stesse condizioni, preservare la partita in corso di svolgimento. Più precisamente, nella Schermata di Gioco deve essere preservata la partita in corso (a) durante un cambio di configurazione e (b) se la schermata è terminata da Android per recuperare risorse, ma non (c) se essa termina per decisione dell'utente. Per il caso (a) è sufficiente verificare il corretto funzionamento durante una transizione portrait/landscape. Per il caso (b) è sufficiente verificare il corretto funzionamento quando la schermata perde il foreground mentre l'opzione di debug "Non conservare attività" è abilitata. Per il caso (c) è sufficiente verificare il corretto funzionamento quando l'utente preme il tasto "Overview" di sistema e poi chiude l'app con uno swipe verso l'alto.

I dati delle partite devono essere memorizzati in maniera persistente in un database SQL, da gestire a scelta con SQLiteDatabase o Room.

## Suggerimenti e note

Riproduzione della sequenza da parte del computer, **temporizzazione**: si suggerisce l'uso di **coroutine Kotlin con la funzione delay**. Quale che sia la soluzione adottata, l'obiettivo è che il **thread principale non rimanga mai bloccato**.

Riproduzione della sequenza da parte del computer, **feedback uditivo**: a seconda dei propri gusti, i toni possono essere (a) generati e riprodotti con AudioTrack; (b) scaricati da un sito come Freesound ([esempio 1](#), [esempio 2](#), [esempio 3](#)), convertiti in PCM e riprodotti con [AudioTrack](#); (c) scaricati da un sito e riprodotti senza conversione con [SoundPool](#).

Nota: l'app funziona anche senza feedback visivi e uditivi. Di conseguenza, un possibile percorso per la realizzazione dell'app è completare prima la logica del gioco, e poi aggiungere i feedback.

Nota: **se la lunghezza massima di una sequenza riprodotta correttamente dal giocatore è  $n$ , la lunghezza della sequenza in cui si è verificato il primo errore è  $n+1$** .

Nota: non ci sono requisiti su cosa succede quando il giocatore preme un rettangolo durante la riproduzione della sequenza da parte del computer.

Nota: non ci sono requisiti in materia di accessibilità, supporto a configurazioni di sistema come il dark mode. Non ci sono impostazioni utente.